INTERNAL TECHNICAL DOCUMENTATION

Ghrab VOC
Vulnerability Operations Center

Architecture, agent design, and technology-stack review of the AI-augmented VOC platform — prepared for the Vulnerability Management team.

Prepared: 16 July 2026

Repository: `voc` · branch `rework`

Scope: backend engine, 6-agent AI layer, REST/SSE API, React SPA, deployment

Audience: VM team leadership + engineers who will maintain/extend the platform

Table of Contents

1 · Executive Summary

The Ghrab VOC is a self-contained, AI-augmented Vulnerability Operations Center. It takes a raw vulnerability scanner export (Qualys-style CSV) and an enterprise architecture document (Markdown CMDB), and produces three layers of value on top of the raw data:

1. **A deterministic, explainable risk score** (the Ghrab Risk Score, or GRS) for every finding — combining CVSS, exploit-probability (EPSS), known-exploitation status (KEV), asset criticality, and blast-radius, gated by network exposure. This requires no AI and no API key.
2. **An autonomous attack-path discovery engine** that builds a reachability graph over the real asset inventory and independently re-derives which findings chain together into an actual internet-to-crown-jewel attack path — it does not read attack paths from a pre-authored script, it computes them from first principles.
3. **A six-agent AI narration/reasoning layer**, built on a vendor-neutral LLM abstraction (litellm) supporting seven providers with automatic fallback, that explains, ranks, cross-references, and briefs on top of the deterministic ground truth — never inventing hosts, CVEs, or connections that aren't in the data.

The system is delivered as two services — a FastAPI backend and a React/TypeScript single-page application — connected by a JSON + Server-Sent-Events API, and ships with Docker Compose for one-command deployment.

~3,200	~4,300	6	7	18
LINES OF PYTHON (BACKEND)	LINES OF TS/TSX (FRONTEND)	AI AGENT MODULES	LLM PROVIDERS WIRED IN	REST + SSE ENDPOINTS

This document is a complete technical map of the platform as it exists today, plus a candid assessment of where the technology stack has gaps that should be closed before this moves from a lab/demo posture toward a production-facing tool. Section 12 is written specifically for that conversation.

2 · What This Platform Is (and Isn't)

The core idea

Most vulnerability dashboards sort a flat list of CVEs by CVSS score. That fails a very common real-world scenario: a CVSS 10 sitting on an isolated, admin-only internal VM is often *less* urgent than a CVSS 7 sitting on an internet-facing box that chains into a database server holding customer records. The Ghrab VOC's central design bet is that **risk is a function of the finding plus where it sits in the network plus what it connects to** — not the finding alone.

It answers this in two independent, verifiable ways:

- **Quantitatively** — the GRS formula (Section 5.1) folds reachability and blast-radius directly into the score, so the ranked findings list already reflects context, before any AI is involved.
- **Structurally** — the attack-path engine (Section 5.3) builds an actual graph of the network and proves, hop by hop, which findings compose into a real attacker path from the internet to a crown-jewel asset.

> **Key architectural principle:** the platform never lets the AI compute or override risk. GRS is plain Python arithmetic; attack paths are graph traversal (NetworkX) over data parsed from the CMDB. The AI agents only ever *narrate, rank, and cross-reference* ground truth that was already computed deterministically — and every AI output is post-parse validated against that ground truth before it reaches the UI (Section 6.4). This is what lets the dashboard run fully, with real risk scores and real attack paths, with **zero API keys configured**.

What it isn't (yet)

- It is **not connected to a live scanner** — it ingests a static CSV export, not a Qualys/Tenable/Rapid7 API.
- It is **not multi-user or authenticated** — there is no login, no per-analyst workspace, no RBAC (Section 12).
- It does **not persist history** — there's no database; state lives in server memory and a single JSON cache file, so there is no trend-over-time view.
- It currently ships with **two synthetic lab datasets** (a fictional financial-services org and a fictional healthcare org), not live organizational data.

3 · Technology Stack at a Glance

Backend

LAYER	TECHNOLOGY	PURPOSE
Web framework	`FastAPI 0.115`	REST + Server-Sent-Events API, auto-generated OpenAPI docs at `/docs`
ASGI server	`uvicorn`	Dev/prod server, hosts the FastAPI app
Data handling	`pandas`	Loads and enriches the vulnerability CSV into a DataFrame — the working data model throughout the backend
Graph engine	`networkx`	Directed reachability graph construction + shortest-simple-paths enumeration for attack-path discovery
LLM abstraction	`litellm ≥1.51`	Single call interface (`litellm.completion`) across 7 model providers — no vendor SDK is called directly anywhere in the codebase
Validation	`pydantic`	Request/response body models for the REST API
Config	`python-dotenv`	Loads provider API keys from a root `.env` file
Runtime	Python 3.12	

Frontend

LAYER	TECHNOLOGY	PURPOSE
Framework	`React 18 + TypeScript 5`	SPA, function components + hooks throughout
Build tool	`Vite 5`	Dev server with API proxy, production bundler
Routing	`react-router-dom 6`	9 top-level routes under a shared `Layout`
Server state	`@tanstack/react-query 5`	Data fetching/caching for every REST call
Tables	`@tanstack/react-table 8`	Sortable findings table
Charts	`recharts 2`	Band-distribution bars, CVSS-vs-GRS scatter, team risk bars
Graph visualization	`cytoscape 3 + cytoscape-fcose + cytoscape-navigator`	Interactive, canvas-rendered attack-surface graph with 3 layout engines, replay, minimap (Section 8.3)
Styling	`Tailwind CSS 3`	Utility-first styling, light/dark theme via CSS custom properties
Motion	`framer-motion`	Micro-interactions / transitions
Icons	`lucide-react`	

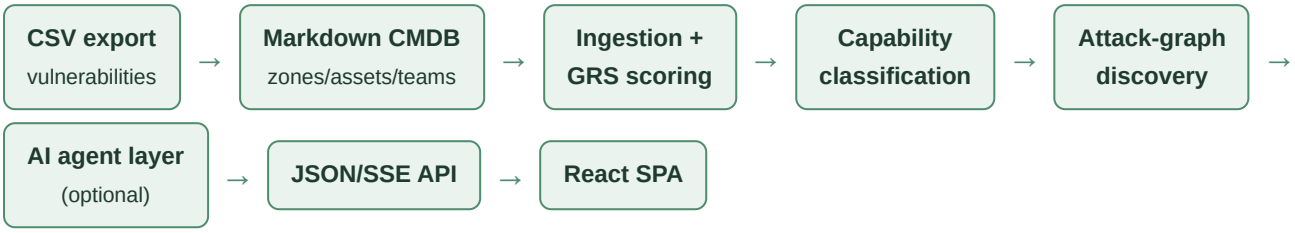
Infrastructure & delivery

CONCERN	APPROACH
Containerization	Two Dockerfiles (Python slim + Node build → nginx static serve); orchestrated with `docker-compose.yml`
Reverse proxy / TLS	nginx in the frontend container terminates TLS, redirects 80→443, serves the SPA, and reverse-proxies `/api/*` to the backend with buffering disabled (required for SSE to flush)
Local dev	`dev.sh` starts both services together; Vite's dev-server proxy handles `/api` without nginx
Persistence	**None** — no database. State is an in-memory singleton (Section 4) plus one JSON file caching AI-layer output (`backend/data/cache/analysis_cache.json`)
Secrets	LLM API keys via `.env` (server-wide) or pasted at runtime into an in-memory settings dict from the Settings screen — session-state wins over env var if both are set

> **Notably absent from the stack:** no database (SQL or NoSQL), no authentication/authorization layer, no message queue, no observability/APM tooling, no CI pipeline, and no live threat-intel integration (EPSS/KEV are hardcoded lookup tables, not API calls). These are addressed as improvement items in Section 12.

4 · System Architecture & Data Flow

The platform is two services talking over HTTP/SSE. There is no shared database between them — the backend is the sole source of truth, computed fresh into memory on first request and cached.



This is the exact sequence run by `agents/orchestrator.py::run_pipeline()`, and it is deliberately **deterministic-first**: the CSV→GRS→capability→graph steps always run, need no API key, and complete in well under a second. Only if at least one LLM provider key is configured does the pipeline proceed into the AI stages (Discovery → Correlation → Compliance → Triage), whose combined output is cached to disk so re-opening the dashboard doesn't re-spend tokens.

Request/response model

There is a single global `AnalysisResult` held in `api/state.py`, computed once and memoised behind a lock. Every REST endpoint reads from this same object; `POST /api/analyze` invalidates and recomputes it, broadcasting live progress to every connected client via SSE — a second browser tab or a page reload attaches to the *same* in-flight run instead of starting a redundant one.

> **Implication:** this is a single-tenant, single-dataset backend process. There is exactly one "current analysis" at a time, shared by every client that connects to it — there is no per-user or per-session isolation. See Section 12.

Live signal channels (SSE)

Three real-time channels run over Server-Sent Events rather than polling:

STREAM	ENDPOINT	PURPOSE
Analysis progress	`POST /api/analyze`	Streams each pipeline stage as it completes ("Ingesting…", "Classifying findings…", "Discovery Agent: validating…", etc.) — drives the progress overlay in the SPA
AI Analyst chat	`POST /api/chat`	Streams the Triage Agent's answer word-by-word for a live-typing feel
Agent activity log	`GET /api/logs/stream`	Tails every LLM call the platform makes in real time — which agent, which provider, success/error/fallback, latency, tokens, cost (Section 6.2)

5 · The Deterministic Engine

Everything in this section runs in pure Python with no network calls and no LLM. It is the trust anchor of the whole platform — the AI layer explains this output, it never computes or overrides it.

5.1 · Ghrab Risk Score (GRS)

Implemented in `backend/core/risk_engine.py`, following the methodology in `ghrab_risk_methodology.md`. It's a deliberate synthesis of five recognized industry approaches (Tenable VPR / attack-path "toxic combinations", FIRST.org EPSS, CISA SSVC's exposure-gating idea, the FAIR risk model's contact-frequency concept, and EU DORA's critical-function weighting) into one composite, auditable number:

```
GRS = min(100, ImpactScore × ExposureTier × CCF)

ImpactScore = 10 × (0.30·CVSS + 0.15·EPSS×10 + 0.15·KEV(0/10) + 0.20·ACW×10 + 0.20·TCM)
```

FACTOR	WEIGHT	SOURCE
CVSS Base Score	30%	As-scanned, from the CSV
EPSS (exploit probability)	15%	Hardcoded per-CVE lookup table (`EPSS_BY_CVE`) — see Section 12 for the "should be live" note
KEV (known exploited, binary)	15%	Hardcoded CISA KEV membership set (`KEV_CVES`)
ACW — Asset Criticality Weight	20%	Hand-derived 0–1 score per finding from the asset's business-criticality tier in the architecture doc
TCM — Toxic Combination Score	20%	Hand-derived 0–10 score reflecting the finding's position in a documented attack chain (terminus steps score highest)

The result is then **gated** — not additively adjusted — by an `ExposureTier` multiplier reflecting network reachability (Internet-facing 1.30 → Adjacent 1.15 → Internal 0.90 → Restricted 0.50 → Isolated 0.15), and by a Compensating Controls Factor (CCF, currently always 1.0 — no verified compensating controls modeled in the lab data). This multiplicative gate is what produces the reordering effect the whole design is built around: an unreachable critical finding gets suppressed toward the bottom of the range, and a reachable moderate finding can be pushed into the danger zone.

ACTION BANDS (SLA)

BAND	GRS RANGE	SLA
IMMEDIATE	80–100	24–72h, emergency change
ACT	60–79	7 days

ATTEND	40–59	30 days
TRACK*	20–39	90 days / next patch cycle
TRACK	0–19	Monitor only

A separate overlay applies for findings flagged as touching an EU DORA "Critical or Important Function" (CIF): the remediation SLA is capped at 30 days regardless of the computed band, reflecting the regulation's own scanning/remediation cadence requirement (RTS Art. 10) for the financial-services dataset.

5.2 · Capability Classification

`backend/core/capability.py` is the first half of the discovery engine. Every finding is mapped — from its own CVSS vector, category, CVE, and free-text description/consequence, using compiled regex keyword signals — into what an attacker actually *gains* from it, expressed in ATT&CK-style terms:

EFFECT	MEANING
`initial_access`	Attacker gains a first foothold from outside a trust boundary
`rce_system / rce_user`	Remote code execution as SYSTEM/root (full host) vs. lower privilege
`credential_theft`	Yields credentials usable elsewhere (with a sub-flag for domain-admin-yielding theft)
`privilege_escalation / cloud_priv_esc`	Elevates locally, or to domain/cloud admin
`segmentation_break`	Opens a network boundary between VLANs/zones — the edge-forming capability the graph engine keys on
`impact`	Terminal business impact (exfiltration, settlement/payment data exposure, etc.)

> **Critical design constraint, stated explicitly in the code comments:** this classification is derived *entirely* from the finding's own content — never from the pre-authored `Attack_Path_Ref` column that documents the "official" attack paths for the lab. That column is reserved purely as an offline oracle for the acceptance test (Section 11). This is what makes the discovery genuinely autonomous rather than a re-narration of a script.

5.3 · Autonomous Attack-Path Discovery

`backend/core/attack_graph.py` composes the per-finding capabilities into a directed reachability graph over the real asset inventory (from the CMDB), then enumerates chains from attacker-reachable entry points to the organization's crown jewels.

GRAPH CONSTRUCTION

- **Entry edges** — INTERNET → any host carrying an internet-reachable, unauthenticated finding.

- **Same-zone lateral movement** — hosts in the same VLAN can reach each other (flat L2 segment assumption).
- **Tier-aware app→db adjacency** — an app server reaches only its *own* database (matched by business-function token in the hostname, e.g. `APP-CRM01 → DB-CRM01`), not the whole DB tier — this avoids fabricating edges between unrelated services that merely share a VLAN.
- **Segmentation-break edges** — a finding that opens a VLAN boundary (or names an explicit DB-link pivot target) adds a directed edge between the two zones/hosts it connects, tagged with the enabling QID.
- **Credential-reuse edges** — shared local credentials link the specifically named hosts, symmetrically.
- **Domain-Admin edges** — a finding that yields Domain Admin (Zerologon, Kerberoasting a DA-cached credential, etc.) adds an edge to *every* AD-joined host in the domain-trust VLANs.

PATH ENUMERATION & SCORING

For each host flagged a **crown jewel** (VLAN in the designated crown zone, a DORA-CIF asset, an Asset Criticality Weight ≥0.88, or a data-impact finding on a database/file-server/backup/SWIFT-labelled host), the engine runs NetworkX's `shortest_simple_paths` from INTERNET, prunes paths through any "empty" intermediate host that has no finding of its own and no specific enabling edge, then scores survivors:

```
score = target_value × 45 + max_GRS × 0.45 + blast_radius × 8 − length_penalty
```

Paths are deduplicated by (entry, target) keeping the best-scoring representative, then **diversified** so every distinct crown-jewel target is represented by at least its best path before the ranked top-14 is filled out by score alone — so a file server, a CRM database, a cloud RDS instance, backups, and the SWIFT gateway all surface as distinct chains rather than the list being dominated by whichever single target has the highest-GRS finding.

5.4 · CMDB Ingestion

`backend/core/cmdb.py` parses the enterprise architecture Markdown document (network zones, asset inventory, teams, and the "official" documented attack-path narrative) via pipe-table regex parsing. It auto-detects between two known document shapes:

- A **legacy prose-table format** (`## 3. Asset Inventory`, `host` / `vlan` / `zone name` columns).
- A newer **CI-based, ServiceNow-style format** (`## 1. CI Inventory`, cross-referenced by CI ID / Rule ID / Relationship ID).

> **Known fragility, already bitten once:** this parser previously failed *silently* on the newer CI-based format — zones/assets/teams all came back empty, starving both the deterministic graph engine and every downstream LLM agent of grounding context, with no error raised anywhere. It was fixed by adding format detection and a second parser path, but the underlying approach — regex-scraping a hand-authored Markdown document — remains structurally fragile to any future document-shape change. See Section 12.

6 · The AI Agent Layer

6.1 · Orchestrator & Pipeline

`agents/orchestrator.py::run_pipeline()` is the single entry point. It is not itself an AI agent — it sequences the deterministic layer, then (only if a provider key exists) calls the AI agents in a fixed order:

Discovery Agent → **Correlation Agent** → **Compliance Agent** → **Triage Agent** executive synthesis

A full run costs roughly 5–6 LLM calls total. The Triage Agent runs *last* deliberately: it is fed a compact digest of what Discovery, Correlation, and Compliance already concluded, and is explicitly instructed to be *consistent* with those prior stages rather than independently re-deriving its own "most urgent finding" — this was a real bug fixed on 2026-07-15 (the dashboard's headline summary could previously contradict the detail panels below it). The Remediation Agent is deliberately **excluded** from this bulk pipeline and only runs on-demand when an operator drills into a specific finding, to keep the one-click "Full Analysis" fast and cheap.

Results are cached to `backend/data/cache/analysis_cache.json` so reloading the dashboard doesn't re-spend tokens; a "Full Analysis" button forces a fresh run.

6.2 · Multi-Provider Architecture (litellm)

No agent ever calls a vendor SDK directly — every call goes through `core/providers.py::call_llm()`, built on litellm's unified `completion()` interface. Each of the five agent *roles* has a preferred provider (deliberately spread across vendors to avoid hammering one bill/rate-limit) plus an automatic fallback chain:

ROLE	DEFAULT PROVIDER	DEFAULT MODEL	USED BY
`attack_path`	Mistral	`mistral-large-latest`	Discovery Agent (narration), Attack Path Agent
`correlation`	Groq	`llama-3.3-70b-versatile`	Discovery Agent (toxic combos), Correlation Agent
`compliance`	Mistral	`mistral-large-latest`	Compliance Agent
`triage`	Mistral	`mistral-large-latest`	Triage Agent (chat + executive synthesis)
`remediation`	Groq	`llama-3.3-70b-versatile`	Remediation Agent

Fallback order if the preferred provider has no key or fails: `mistral → groq → gemini → anthropic → openai → deepseek → xai`. All seven providers are fully wired (env var, default model, litellm prefix) in `core/config.py`, and every one is overridable per-role from the Settings screen at runtime.

CLIENT-SIDE RATE LIMITING

Added 2026-07-14 after a full pipeline run (≈6 back-to-back calls) was tripping free-tier requests-per-minute caps. `core/providers.py` now:

- Spaces consecutive calls to the *same* provider by a minimum interval (thread-safe, since calls run on background threads for SSE handlers) — tuned per provider: Groq 2.5s (~30 RPM tier), Gemini 4.5s (~15 RPM tier), Mistral 1.5s, others default 4.0s.
- On a 429, retries the *same* provider with exponential backoff (honoring a `Retry-After` header if present) up to 4 attempts, before falling through the vendor chain — reasoning that the next provider in the chain is on an equally thin free tier, so burning straight through it just spreads the failures around instead of avoiding them.
- All of this is tunable via environment variables with zero code change, and can be disabled entirely (interval=0) for a paid tier.

LIVE OBSERVABILITY — THE AGENT ACTIVITY LOG

`core/call_log.py` keeps an in-memory, pub-sub log of *every* LLM call attempt — start, success, or error — including provider, model, duration, prompt/completion tokens, and best-effort USD cost (via litellm's pricing table). This feeds the "Agent Logs" screen in the SPA (Section 8), letting an operator see live which agent is calling which provider right now, and audit why a fallback happened. It is deliberately **not persisted** — cleared on backend restart (flagged in Section 12).

6.3 · The Six Agents, In Detail

① DISCOVERY AGENT — `AGENTS/DISCOVERY_AGENT.PY`

The reasoning layer directly on top of the deterministic path engine. Runs in two bounded passes (works across every provider without vendor-specific tool-calling):

1. **Narration pass** — for every candidate chain the graph engine found, produce a headline, a 3–5 sentence hop-by-hop narrative, business impact, the single choke-point QID+host that breaks the chain, a confidence rating (high/medium/low), and a novelty rating (textbook/notable/non-obvious).
2. **Toxic-combination pass** — reason *beyond* the enumerated chains to surface up to 4 cross-path risks an operator scanning the list top-to-bottom would miss (shared credentials, flat trusts merging otherwise-separate chains), plus a 3–4 sentence executive synthesis of overall attack-surface posture.

It is explicitly instructed never to invent hosts or QIDs — the candidate chains are handed to it as ground truth.

② CORRELATION AGENT — `AGENTS/CORRELATION_AGENT.PY`

Cross-references the *entire* findings table against the CMDB (bounded to standalone/non-chained findings, so it doesn't re-derive what the deterministic graph already proves) to surface: (1) cross-finding risk not already captured in a documented chain, only if it can point to a real mechanism (shared asset/team/credential/network adjacency); (2) the top risk-owning teams by aggregate exposure; (3) findings it believes are mis-prioritized relative to their true blast radius.

③ COMPLIANCE AGENT — `AGENTS/COMPLIANCE_AGENT.PY`

Reasons over the `Compliance_Ref` column plus the CMDB's regulated-scope facts (e.g. PCI CDE, SWIFT CSP scope, DORA critical-function status) to produce a posture briefing: frameworks in scope, key gaps (each grounded to specific finding QIDs), a note on what the DORA CIF SLA overlay means operationally, and a 3–4 sentence auditor/board-readable executive summary. It is explicitly framed to *flag gaps* rather than assert formal certification status the platform has no authority to know.

④ TRIAGE / ANALYST AGENT — `AGENTS/TRIAGE_AGENT.PY`

Powers two surfaces: the free-form **AI Analyst chat** (grounded Q&A over the full CMDB + findings table + discovered attack paths, with conversation history) and the **one-paragraph executive synthesis** on the Command Center. As the pipeline's last stage, it is fed a compact digest of what the three prior agents already concluded and instructed to anchor its "most urgent finding" and "most exposed team" on that prior work rather than compete with it (see Section 6.1).

⑤ REMEDIATION AGENT — `AGENTS/REMEDIATION_AGENT.PY`

Called *on-demand* only, when an operator drills into a single finding — not part of the bulk pipeline, to keep the one-click "Full Analysis" fast and cheap. Enriches the scanner's raw remediation text into a plain-English risk summary for non-technical stakeholders, an ordered step-by-step remediation procedure, validation steps, a note on any operational risk the fix itself introduces, and an effort estimate (Low/Medium/High).

⑥ ATTACK PATH AGENT — `AGENTS/ATTACK_PATH_AGENT.PY`

Distinct from the Discovery Agent's bulk narration: this one narrates a *single*, already-built deterministic chain (from `core/graph.py`'s `Chain` — built from the `Attack_Path_Ref` oracle column) into headline / step-by-step narrative / business impact / primary choke point / owning teams. The step sequence itself is ground truth and is never invented.

6.4 · Anti-Hallucination / Grounding Design

Every JSON-mode agent output is **post-parse validated** against the deterministic ground truth before it reaches the API/UI — this is applied uniformly, at zero extra LLM-call cost:

- **Discovery Agent:** any narrated `path_id` not present in the graph engine's actual candidate set is dropped and reported as `hallucinated_path_ids`; any `choke_point` whose text doesn't reference a real QID is flagged `choke_point_unverified` rather than presented with unwarranted confidence; toxic combinations citing zero real QIDs are dropped entirely, others have their QID list filtered to only verified ones.
- **Correlation Agent:** reprioritization flags citing a QID not in the findings table are dropped, with a `dropped_reprioritization_flags` count surfaced.
- **Compliance Agent:** gaps citing zero real QIDs are dropped entirely (an ungrounded compliance claim is worse than none); others keep only their verified QID references.

Additionally, every agent shares one system-prompt persona (`agents/base.py::ANALYST_PERSONA`) that explicitly instructs: reason only from the supplied context, never invent CVEs/hosts/teams/connections, and say so explicitly when not confident rather than guessing.

7 · API Surface (REST & SSE)

All endpoints are prefixed `/api`. Full interactive documentation is auto-generated by FastAPI at `/docs`.

METHOD & PATH	PURPOSE
`GET /health`	Liveness check
`GET /overview`	Command Center payload: KPIs, executive summary, CVSS-vs-GRS scatter data, top 5 findings, top 5 attack paths, action-band table
`GET /kpis`	Just the KPI tile numbers
`GET /findings`	Full findings list, GRS-sorted
`GET /findings/{qid}`	Single finding detail incl. GRS factor breakdown
`POST /findings/{qid}/remediation`	Triggers the Remediation Agent for one finding (requires AI provider — 409 otherwise)
`GET /attack-paths`	Discovered + narrated paths, toxic combinations, documented-path overlay
`GET /graph`	Full node/edge payload for the Cytoscape graph
`GET /correlation`	Correlation Agent output
`GET /compliance`	Compliance Agent output
`GET /teams`	Per-team exposure rollup
`GET /settings/providers`	Provider configuration state + current role→provider assignments
`POST /settings/providers`	Set/clear an API key for a provider (invalidates the cached analysis)
`POST /settings/agent`	Override which provider serves a given agent role
`GET /analyze/status`	Lets a freshly loaded page discover an already-in-flight run
`POST /analyze` **SSE**	Kicks off (or attaches to) the full pipeline, streaming stage-by-stage progress
`POST /chat` **SSE**	AI Analyst chat — streams the Triage Agent's answer
`GET /logs · DELETE /logs`	Full LLM call history / clear it
`GET /logs/stream` **SSE**	Live tail of every LLM call, with an authoritative "currently active" resync snapshot on each (re)connect

8 · Frontend Application

A single-page "command-center" style UI, organized as 9 routed screens under a shared sidebar/header layout (`src/App.tsx` , `src/components/Layout.tsx`):

SCREEN	ROUTE	WHAT IT SHOWS
Command Center	`/`	KPI tiles, executive synthesis, band-distribution + CVSS-vs-GRS charts, most-urgent findings, top attack paths
Findings	`/findings`	Sortable/filterable/searchable table (TanStack Table) of every finding; click-to-drill slide-over detail drawer
Attack Paths	`/attack-paths`	The interactive attack-surface graph + ranked path rail + toxic combinations (Section 8.1 below)
Correlation	`/correlation`	Cross-finding insights, top risk-owning teams, reprioritization flags
Teams	`/teams`	Per-team peak-risk bar chart + exposure cards, click-through to filtered findings
Compliance	`/compliance`	Frameworks in scope, DORA CIF overlay note, key gaps grounded to QIDs
AI Analyst	`/analyst`	Streaming chat grounded in live findings/CMDB/attack-path data, with suggested prompts
Agent Logs	`/logs`	Live + historical feed of every LLM call, filterable, exportable to CSV
Settings	`/settings`	Per-provider API key entry, per-role provider assignment

Every screen with meaningful content offers a one-click **Markdown/CSV export** (`src/lib/report.ts` , `reportBuilders.ts`), plus a header-level "Full Report" action that assembles overview + findings + attack paths + correlation + teams + compliance into a single downloadable Markdown briefing.

8.1 · Attack Graph Visualization

The attack-path screen's centerpiece (`features/attackpaths/AttackGraph.tsx` , ~880 lines) is a custom Cytoscape.js canvas renderer — not a generic chart library — purpose-built for this data:

- **Three layout engines**, switchable live: *Flow* (BFS-distance columns from INTERNET), *Zones* (grouped by network zone with compound "lane" containers), *Radial* (concentric rings by hop-distance).
- **Path replay** — selecting a discovered path reveals it hop-by-hop with a play/pause/scrub control, animating each new edge into view.
- **Docked node inspector** — hover or click any host to see its role, owning team, peak GRS + remediation SLA, target-value score, finding count, and connection degree, without a floating tooltip obscuring the graph.
- **Edge-kind filtering** — toggle entry/segmentation/credential/domain/lateral edges independently to declutter a dense graph.
- **Minimap, fullscreen, search-to-highlight, and full light/dark theme support** — the stylesheet is rebuilt from the DOM's resolved CSS custom properties whenever the theme toggles, since

Cytoscape draws on a `<canvas>` and cannot resolve CSS variables itself.

8.2 · Live Agent Activity

A pill in the header (`Layout.tsx::AgentActivityPill`) shows, at a glance, whether an agent is currently calling a provider — clicking it jumps to the full Agent Logs screen. This is backed by a persistent SSE subscription (`lib/agentLog.tsx`) to `/api/logs/stream` that survives route navigation, so activity is visible from anywhere in the app.

9 · Multi-Tenant Lab Datasets

The platform ships with two complete, self-consistent synthetic datasets, each with its own CSV, architecture Markdown, and org profile (name/sector/compliance frameworks) that the AI analyst persona adapts to automatically:

DATASET	SECTOR	FRAMEWORKS	FILES
Ghrab Financial Group	Financial services	PCI DSS, SWIFT CSP, EU DORA	`ghrab_vulnerabilities(_v2).csv`, `ghrab_architecture(_v2).md`
Velon Health Systems	Healthcare provider	HIPAA Security Rule, FDA Premarket/Postmarket Cybersecurity Guidance	`velon_vulnerabilities.csv`, `velon_architecture.md`

> **Currently active dataset:** `backend/core/config.py` currently points `VULN_CSV_PATH` / `ARCHITECTURE_MD_PATH` at the **Velon Health Systems** files (the Ghrab paths are present but commented out just above). Anyone reading the dashboard today is looking at the healthcare lab, not the financial-services one the platform's own README describes as the primary scenario — worth confirming this is the intended demo state before presenting.

Switching datasets is a two-line config edit; the risk-engine's per-finding factor tables (`FINDING_FACTORS`, `EPSS_BY_CVE`, `KEV_CVES` in `risk_engine.py`) are QID/CVE-keyed and simply unioned across both labs since the two datasets' identifier namespaces don't collide — there's no "active lab" branching logic in the scoring code itself.

10 · Deployment & Operations

Local development

Backend	`cd backend && uvicorn main:app --reload --port 8000`
Frontend	`cd frontend && npm run dev`
Both at once	`./dev.sh`

Docker

`docker compose up --build` starts two services:

- **backend** — Python 3.12-slim, installs `requirements.txt`, runs uvicorn on :8000; the analysis cache directory is bind-mounted so it survives container restarts.
- **frontend** — multi-stage build (Node 22 → static Vite bundle served by nginx 1.27-alpine); nginx redirects HTTP→HTTPS, terminates TLS from a mounted cert pair, serves the SPA with client-side-routing fallback, and reverse-proxies `/api/` to the backend with buffering disabled so SSE frames flush immediately instead of batching.

Provider API keys are supplied via a root `.env` file (see `.env.example` — all seven keys are optional; the platform runs fully deterministic with zero configured).

> **No CI/CD pipeline exists.** There is no `.github/workflows` directory, no automated build/test/lint gate on push or PR, and no automated Docker image publishing. Every deploy today is a manual `docker compose up --build` on the target host.

11 · Testing & Quality Assurance — Current State

There is exactly one automated test in the entire codebase: `backend/tests/test_rediscovery.py`. It is a genuinely meaningful **acceptance gate** — it runs the full deterministic pipeline (ingestion → capability classification → graph discovery) and checks that the engine's own reachability graph independently proves a path from INTERNET to each documented attack path's entry and target host, using the hand-authored `Attack_Path_Ref` column strictly as an offline oracle it never feeds into the engine itself. At last verification this passes **6/6** on both shipped datasets.

Beyond that single test, there is no automated test coverage anywhere in the platform:

- No unit tests for the GRS formula (`risk_engine.py`) — the exact arithmetic that is the platform's core trust claim is unverified by any test that would catch a regression.
- No unit tests for capability classification edge cases (the regex keyword signals in `capability.py` are intricate and easy to silently break).
- No API/integration tests for any of the 18 REST/SSE endpoints.
- No frontend tests of any kind — no component tests, no E2E tests. Zero `*.test.*` / `*.spec.*` files exist in `frontend/src` .
- No CI workflow runs even the one existing test automatically on push/PR.

12 · Recommended Improvements to the Tech Stack

These are gaps identified directly from reading the implementation, ranked by how much they matter before this platform could responsibly hold real organizational vulnerability data or be exposed beyond a single trusted operator's laptop.

Security & multi-user readiness

No authentication or authorization anywhere in the API HIGH

Every endpoint — including `POST /settings/providers`, which accepts and stores raw LLM API keys — is open to any client that can reach the backend. There is no login, no session, no API token, no RBAC. Before any deployment beyond a single trusted operator's machine, this needs at minimum an auth middleware (even a simple bearer-token gate) in front of the FastAPI app.

Secrets held in plaintext, in-memory, server-wide HIGH

`api/state.py::runtime_settings` is a single process-wide dict — any operator's pasted API key is visible to (and usable by) every other client hitting the same backend, with no encryption at rest and no secrets-manager integration (Vault, AWS Secrets Manager, etc.). Combined with the lack of auth above, this is the same risk twice.

No rate limiting on cost-incurring endpoints MEDIUM

`POST /analyze` and `POST /chat` both trigger real LLM spend. With no auth and no rate limit, a single misbehaving client (or a stray automated retry loop) can run up a provider bill with no platform-level guardrail beyond the per-provider throttling that exists purely to avoid 429s, not to control cost.

Data & persistence

No database — single global in-memory analysis state HIGH

`api/state.py` holds exactly one `AnalysisResult` for the whole process. This means: no multi-tenant/multi-dataset support even though the codebase already supports switching lab profiles by config edit; total state loss on every backend restart; and no way to view risk trending over time, since nothing about a prior analysis run is retained once superseded. A real deployment needs at least a lightweight database (Postgres, or even SQLite to start) to persist findings history, analysis runs, and enable trend/delta reporting — something a Vulnerability Management team will very quickly ask for ("is this getting better or worse month over month?") and the current architecture structurally cannot answer.

EPSS and KEV data are hardcoded lookup tables, not live feeds HIGH

`risk_engine.py`'s `EPSS_BY_CVE` and `KEV_CVES` are hand-populated dictionaries covering only the ~60 CVEs used across the two lab datasets — this is explicitly flagged in the code's own docstring as a lab shortcut. For any real dataset, GRS silently falls back to `EPSS=0.0` for every unrecognized CVE, which quietly zeroes out 15% of the score's weight. This must become a live integration against FIRST.org's EPSS API and CISA's published KEV catalog JSON before the platform can score real-world findings correctly.

CMDB ingestion is regex-parsed Markdown, proven fragile MEDIUM

As documented in Section 5.4, this exact parser has already silently failed once (zero-asset parse on a differently-shaped architecture doc), starving the entire platform of grounding with no error surfaced. A structured input format — a JSON/YAML CMDB export, or a real integration against a CMDB system of record (ServiceNow, device42, etc.) — would remove an entire class of "looks fine, silently produces empty analysis" failure that plain-text scraping can never fully guard against.

No live scanner integration MEDIUM

The platform ingests a static CSV snapshot. Operationalizing this against Qualys/Tenable/Rapid7's actual export or API would remove the manual "drop a new CSV in `backend/data/`" step and is a natural next integration once persistence (above) exists to make repeated ingestion meaningful.

AI / LLM layer

Correlation and Remediation default to the weakest model for arguably the hardest reasoning task MEDIUM

Per `core/config.py`, `correlation` and `remediation` roles default to Groq's `llama-3.3-70b-versatile` (the fastest/cheapest model in the roster), while cross-referencing the full CMDB and findings set to find non-obvious risk is arguably the platform's hardest reasoning task. This was a deliberate rate-limit-spreading tradeoff, not an accident, but it's an open question worth revisiting now that grounding/verification (Section 6.4) has reduced the cost of a wrong answer — a stronger default model here may now be worth the latency/cost tradeoff.

Chat "streaming" is simulated, not real token streaming LOW

`api/streams.py::chat()` waits for the full Triage Agent response, then re-emits it word-by-word with a fixed 12ms delay to fake a live-typing effect. litellm supports real provider-side streaming; wiring that through would meaningfully cut perceived time-to-first-token on longer answers, since today the user waits for the entire generation before seeing anything.

Hardcoded default model IDs will drift stale LOW

Model identifiers like the current defaults are pinned directly in `core/config.py` with no update mechanism — as providers deprecate model versions, these will silently start failing or serving worse fallback models with no platform-level alert. Worth a periodic review process or a config surface that doesn't require a code change to bump.

Agent call history is not persisted LOW

`core/call_log.py` is explicitly in-memory only, capped at 300 entries, and wiped on restart. For a platform that makes AI-assisted risk and compliance judgments, an audit trail of every AI call (what was asked, what was answered, which provider/model, when) is a reasonable compliance expectation that the current design cannot meet.

Testing, CI/CD & observability

Near-zero automated test coverage HIGH

As detailed in Section 11: one acceptance test exists, covering only path rediscovery. The GRS formula, capability classification, every API endpoint, and the entire frontend have zero automated tests. This is the single highest-leverage investment before further feature work — regressions in the risk math or the parsers are currently only caught by manual inspection.

No CI/CD pipeline MEDIUM

No GitHub Actions (or equivalent) exists to run tests, lint, type-check, or build images automatically. Combined with the testing gap above, nothing currently prevents a broken change from reaching `main`.

No production observability MEDIUM

No structured logging, error tracking (e.g. Sentry), or metrics/tracing (e.g. OpenTelemetry/Prometheus) exists anywhere in the stack. The only operational visibility today is the Agent Activity Log, which covers LLM calls specifically, not general application health, request latency, or error rates.

Frontend

No route-level code splitting LOW

`src/App.tsx` statically imports all 9 feature screens, so the initial bundle includes Cytoscape, Recharts, TanStack Table, and every screen's code regardless of which route the user lands on first. Lazy-loading routes with `React.lazy` would reduce first-paint bundle size, particularly since Cytoscape + its layout plugin are a non-trivial chunk of weight.

13 · Glossary

TERM	MEANING
GRS	Ghrab Risk Score — this platform's composite, deterministic risk score (Section 5.1)
CVSS	Common Vulnerability Scoring System — the industry-standard technical-severity score, one input to GRS
EPSS	Exploit Prediction Scoring System (FIRST.org) — probability a CVE is exploited in the next 30 days
KEV	CISA's Known Exploited Vulnerabilities catalog — binary "is this being exploited in the wild right now" gate
ACW	Asset Criticality Weight — how much a given asset matters to the business/regulatory scope, independent of any specific finding
TCM	Toxic Combination Score — how much worse a finding is because of what it connects to, not just what it is
CCF	Compensating Controls Factor — discount for verified mitigating controls (currently unused, always 1.0)
DORA / CIF	EU Digital Operational Resilience Act; "Critical or Important Function" — a regulated-scope flag driving the 30-day SLA overlay
Crown jewel	A host the discovery engine has identified as a high-value attack target (high asset-criticality, DORA-CIF scope, or terminal data-impact finding)
Toxic combination	A cross-finding risk pattern (shared credential, flat trust, etc.) that isn't visible from any single finding in isolation
litellm	The open-source Python library providing one call interface across many LLM vendors — the platform's entire multi-provider abstraction
SSE	Server-Sent Events — the one-way streaming HTTP protocol used for live progress, chat, and log tailing in this platform

---

Prepared by reading the live codebase at `/home/salamnki/Documents/voc` (branch `rework`) — every claim in this document traces to a specific file cited inline. No claim here is inferred from documentation alone; all descriptions of behavior were verified against the actual implementation.